

TC-EFT: Characterizing and Correcting Tensor Core Arithmetic

Anonymous Author(s)

Abstract—Tensor Cores accelerate matrix multiplications within GPUs by rapidly computing expressions of the form $D = A \cdot B + C$, where A, B, C are small floating-point matrices. In addition to AI and ML that drove their rapid development and adoption, many accuracy-critical applications have begun exploiting the speed advantages of Tensor Cores. Their numerical behavior differs across instruction paths and generations.

In this paper, we present an error-free decomposition for recovering IEEE-defined correctly rounded results from the computed $D[i, j]$ entries and their original operands. We use a model that preserves raw product significands and exponent sums, distinguishes alignment padding from a second product integer bit, and specifies exponent floors and output conversion. We bound both alignment and output errors and derive a realizable family of non-monotonicity examples: decreasing an accumulator input can increase the computed output.

Our major contribution is a Tensor Core error-free transformation (TC-EFT) expressed through exact residuals. We prove an equivalent signed overlap identity and sufficient conditions for consolidating its components with scalar additions and one final nearest-even rounding. These results are parametric over specified globally aligned FP32-output profiles on V100, A100, and H100/H200. An exact reference implementation and independent dyadic oracle exercise the corrected model, while the initial 100-case V100 experiment is retained as a separate historical observation.

This work can help convert differing Tensor Core answers into a single correctly rounded canonical answer for the same exact dot product. It also provides a basis for checking software portability against explicit numerical models. Exposing raw product metadata and retained sums could reduce recovery work, but the reference construction does not establish the low-cost overlap extractor or a universal padding threshold for monotonicity. Given the growing importance of Tensor Cores in ML and HPC, these precise contracts support further development of efficient, justified correction algorithms.

I. INTRODUCTION

Modern GPUs employ Tensor Cores as one of the central primitives within them to accelerate matrix multiplication. Tensor Core operations can be requested through PTX matrix instructions, which are lowered to device instructions such as HMMA [1, 2]. For example, a PTX `mma.sync.aligned` operation with shape `m16n8k8`, FP16 multiplicands, and FP32 accumulation obtains a floating-point 32-bit (FP32) matrix $D[16 \times 8]$ by multiplying two FP16 matrices $A[16 \times 8]$ and $B[8 \times 8]$, and adding it to an FP32 matrix $C[16 \times 8]$. The reason for adding C is to allow a series of AB products to be added, thus iterating over a (much) larger matrix at a high sustained pipelined rate. Matrix multiplication is the backbone of AI and ML; in this domain, Tensor Cores offer from $4 \times$ speedups in training trillion-parameter generative AI models to a $30 \times$ increase in inference performance [3, 4]. This potential

eggs even the community that requires more accuracy-critical matrix multiplication to embrace them: extended precision scientific computing [5], an algebraic multigrid solver [6], implementing arithmetic reductions [7], breadth-first search [8], offering FP32 accuracy using FP16 [9], FFT [10], mixed-precision linear algebra [11], and quantum simulation [12] to name a few.

Unfortunately, Tensor Core instruction paths can behave numerically differently [13, 14]. Manufacturer documentation specifies formats and some precision requirements, but leaves accumulation order and rounding unspecified for these mixed-precision paths [1], unlike the scalar IEEE floating-point contract [15]. While the AI/ML community may handle this knowledge-gap by adjusting training and inference methods, accuracy-critical (especially HPC) systems do not have this ability. Not having a rigorous understanding of this important GPU subsystem prevents software libraries using Tensor Cores from having a formal specification or being reliably ported across models.

Beginning with the work in [16], the community has begun piecing together the behavior of Tensor Cores through systematic testing and reasoning about the produced outputs. This work was extended in [13] and [17] where many more critical Tensor Core features—such as the number of “hidden padding bits” (extra precision bits that play a significant role in Tensor Core behaviors), addition order, type of rounding, and many more—were discovered. The refined models of Khattak and Mikaitis [18] additionally identify unnormalized raw products and alignment-exponent floors. We use their v4 specification for the paths studied here.

Even with all these developments, there remained two crucial questions addressed by this work under explicit model assumptions: *Correct Rounding through Error-Free Transformation*, and *Genesis and Extent of Non-Monotonicity*. These are now described followed by a roadmap.

a) *Correct Rounding Through Error-Free Transformation*: Each $D[i, j]$ produced by a Tensor Core consists of $C[i, j]$ added to the k products $A[i, k] \cdot B[k, j]$, where the products are known to be exact (no loss of precision) because Tensor Cores guarantee sufficient precision for computing the products. For example, FP16 significand products need at most 22 bits. Their raw significand products and exponent sums remain paired without normalization; exactness does not mean that every BF16 or TF32 product is an encoded FP32 intermediate [18]. The main loss of precision occurs in the manner in which these products (along with C , which is also FP32) are added by the internal *multi-term adder* [19] of a

Tensor Core. More precisely:

- 1) How far off is $D[i, j]$ with respect to a *correctly rounded* [15, 20] version (return the representable floating-point number that is nearest to the result of the $(k + 1)$ -term addition conducted in infinite precision, with ties to even). We denote this by $D[i, j]_{CR}$.
- 2) How can one *correct* the $D[i, j]$ to obtain $D[i, j]_{CR}$ through extra GPU computational steps?

Knowing these answers allows programmers using Tensor Cores in accuracy-critical applications to obtain the ground-truth answer that everyone in formal floating-point reasoning seeks (namely, correctly rounded answers). They can also obtain the ground-truth answer after porting to a different Tensor Core. This is because our work recovers this ground-truth *for the globally aligned FP32-output profiles and recovery conditions specified here* through an *Error-Free Transformation* (EFT, [21, 22]) approach that accepts the input format, raw-product representation, alignment policy, instruction grouping, and output stages as parameters. Even after five decades, EFT is an important topic in floating-point reasoning (e.g., [23–25]). TC-EFT specializes error-free decomposition to these Tensor Core models. Its exact reference construction and conditional scalar implementation are distinguished from the still-unproved low-cost overlap optimization.

b) Genesis and Extent of Non-Monotonicity: It has been known for some time [14, 16] that when one of the $(k + 1)$ summands involved in obtaining $D[i, j]$ (namely $C[i, j]$ and all the k products $A[i, k] \cdot B[k, j]$) is *increased*, the sum *decreases*. Given that this archetypal non-monotonic behavior can undermine delicate assumptions made in numerical codes [14], it is imperative that we know *the exact conditions under which* non-monotonicity sets in and *to what extent it can skew answers*. The fundamental reasons for non-monotonicity, when formally captured, provides us with a solid foundation for understanding why $D[i, j]$ differs from $D[i, j]_{CR}$. For this reason, we first present results pertaining to non-monotonicity followed by techniques to recover $D[i, j]_{CR}$ from $D[i, j]$.

The manner in which these adds “at once” add $(k + 1)$ elements before normalization changes the analysis: (1) alignment and truncation can lead to non-monotonicity, and (2) a two-term EFT such as Fast2Sum [24] does not directly recover the residual of this block operation.

We show how *the number of terms in the block adder and the number of padding bits*, together with the actual alignment grid and output quantization, underlie a realizable family of non-monotonicity examples and the design of TC-EFT. Padding bits are bits of lower significance lying below the least significant bit of the standard 23-bit (in single-precision) mantissa that many Tensor Cores have.¹ We also discuss how the mantissa width and the range over which the exponents fall affect the manner in which (and the cost through which) TC-EFT can be realized. When these padding bits are present,

¹We later explain how padding bits differ from the trio of bits called *guard* (*G*), *rounding* (*R*) and *sticky* (*S*) that standard arithmetic units have—the backbone of how CPUs and GPU SIMT cores (for instance) support correct rounding.

Tensor Cores retain additional low-order contributions. Their effect on distance to $D[i, j]_{CR}$ depends on signed cancellation and the final truncation. For error correction, we need to avoid “double-counting” contributions already included in the Tensor Core result. This requires the raw alignment exponent, which can differ from the normalized exponent of the largest product. While manufacturers do not document these padding bits, their presence is well-known and their number is straightforward to extract through recently proposed testing-based methods [13, 16, 17].

c) Contributions: In §III, we bound alignment and output errors and give a realizable construction of non-monotonicity with explicit padding and term-count conditions. Then in §III-C, we derive the maximum input decrease and maximum output increase separately for that construction. In §IV, we construct the components needed to recover the exact block sum. We then describe the conditions §IV-B under which $D[i, j]_{CR}$ can be recovered from $D[i, j]$ and the original operands using exact components and a justified final rounding step. Then §V describes the reference implementation, exact model checks, and historical proof-of-concept hardware observations, related work (§VI), discussions and conclusions (§VII, §VIII).

II. PRELIMINARIES

We recap floating-point arithmetic (§II-A) and EFT (§II-A1), and basics of Tensor Cores (§II-B).

A. Floating-Point Basics

Let $\mathbb{R}$ be the set of real numbers, $\mathbb{N}$ the set of natural numbers, and $\mathbb{Z}$ the integers. We write $\mathbb{F}_{sb}^{eb} \subset \mathbb{R}$ to denote the finite values of a binary format with eb exponent bits and sb significand bits. $\mathbb{F}^{32}$ and FP32 are shorthand for $\mathbb{F}_{24}^8$; $\mathbb{F}^{16}$ and FP16 for $\mathbb{F}_{11}^5$; and $\mathbb{F}^{64}$ and FP64 for $\mathbb{F}_{53}^{11}$. An encoding and its real value are distinct: $\text{val}(x)$ denotes the value of an encoded input or output. We usually write x for this value in real-arithmetic expressions.

Definition II.1 (Floating-Point Representation). An encoded floating-point term has a sign $sign \in \{0, 1\}$, an unsigned exponent field, and an $(sb - 1)$ -bit fraction m . With bias $b = 2^{eb-1} - 1$, the unbiased normal exponent satisfies $e_{\min} = 1 - b \leq e \leq e_{\max} = b$. A normal term has value

$$x = (-1)^{sign}(1.m)2^e. \quad (1)$$

The significand $1.m$ has sb bits, including the implicit leading bit. A subnormal instead has value $(-1)^{sign}(0.m)2^{e_{\min}}$; its implicit bit is zero. A zero fraction and exponent field encode signed zero.

For a normal term with exponent e , $\text{ULP}(x)$ is the weight of the least significant fraction bit:

$$\text{ULP}(x) = 2^{e-(sb-1)}. \quad (2)$$

For example, $x = 2 + 2^{-22}$ in FP32 has exponent 1 and $\text{ULP}(x) = 2^{-22}$. For subnormal or zero x , we use the constant quantum $q_{\min} = 2^{e_{\min}-(sb-1)}$ (FP32: 2^{-149}). At a normal

power of two, the gap to the predecessor may be half the gap to the successor.

NaNs and infinities are excluded from the real-valued results below. Finite inputs alone do not guarantee a finite accumulated output. The error bound assumes $|A_{\text{acc}}| \leq \text{maxFinite}_{32}$; each scalar exactness result also states its own range and subnormal-grid requirements. Input subnormals use the decoder in §II-B. We compare the values of signed zeros unless an encoding rule is explicitly stated.

Definition II.2 (Correct Rounding). For an exact result r in the finite range of the target format, the correctly rounded result is

$$y = \text{RN}_{\text{even}, \mathbb{F}}(r). \quad (3)$$

This selects the nearest representable value. If two adjacent values are equally close, it selects the one with an even least significant significant bit. Gradual underflow uses the same rule, with zero as a possible neighbor. Exact zero is encoded as $+0$ in our reference implementation. A half-ULP inequality alone does not specify this operation.

We use $\mathcal{E}$ to emphasize exact real summation. Ordinary $+$, $-$, and $\sum$ in mathematical identities also denote real arithmetic; rounded scalar operations are explicitly written $\oplus$ or $\ominus$, with their format and rounding mode specified.

1) *Recap of Standard Error-Free Transformations (EFT)*:

We now give a short example of EFT: Fast2Sum [24]. For binary floating-point x, y with $|x| \geq |y|$, execute $R = x \oplus y$, $t = R \ominus x$, and $e = y \ominus t$ in the same precision using round-to-nearest ties-to-even. Under the standard no-overflow and gradual-underflow hypotheses, this returns an expansion (R, e) satisfying

$$\text{val}(R) + \text{val}(e) = x + y$$

in exact arithmetic. A further rounded addition $R \oplus e$ need not return the exact real sum. The result of this EFT will be denoted (R, e) in §IV.

B. Tensor-Core Basics

Tensor Cores were introduced in the Volta architecture [26]. We consider the globally aligned FP32-output paths for FP16, BF16 ($\mathbb{F}_8^8$), and TF32 ($\mathbb{F}_{11}^8$, also called tf19) documented in Accurate Models of NVIDIA Tensor Cores, v4 [18]. These use a bespoke accumulation scheme, deviating from methods familiar to the IEEE-754 community [16, 17].

Within a Tensor Core, products may remain unnormalized during computation. The extended alignment format (“ExtFormat”) has two integer magnitude bits and $F = 23 + p$ fractional bits, where p counts the additional low-order padding bits. The second integer bit accommodates a significant product in $[2, 4)$; it is distinct from the p fractional bits. Signs and accumulation carries require separate representation.

For each cell, define the ideal dot-product-add and computed output separately:

$$S_{i,j} = C_{i,j} + \sum_{x=1}^K A_{i,x} B_{x,j}, \quad D_{i,j} = \text{TC}_\theta(A, B, C)_{i,j}. \quad (4)$$

TABLE I: Selected FP32-output profiles from [18, Table 3]. W is the reference’s accumulation-width label, before choosing a software sign convention.

GPU	Input	K	F	W
V100	FP16	4	23	28
A100	FP16/BF16	8	24	30
A100	TF32	4	24	29
H100/H200	FP16/BF16	16	25	32
H100/H200	TF32 (maximum block)	8	25	31

Here K is the number of products accumulated before a single normalization, and $n = K + 1$ includes $C_{i,j}$. This block width is not a square matrix dimension. A PTX instruction or WMMA operation can comprise several such blocks, with intermediate outputs fed back as C . The parameter θ records the input format, instruction grouping, alignment layout, exponent policy, and output stages. For brevity, one cell’s $S_{i,j}$ and $D_{i,j}$ are written S and D .

Some Hopper TF32 WMMA paths have $K = 4$, while the documented `mma.sync.aligned.m16n8k8` path has $K = 8$ [18, Tables 3–4]. Turing is not characterized by that reference and is not assigned a profile here.

Here is how the selected Tensor Cores compute their output:

- 1) **Pairwise Multiplications:** Decode each nonzero operand as $a = (-1)^{s_a} \mu_a 2^{e_a}$. For normal inputs $1 \leq \mu_a < 2$; for subnormals $0 < \mu_a < 1$ and $e_a = e_{\min}$. Form the exact raw product

$$(s_a \text{ xor } s_b, \mu_a \mu_b, \rho = e_a + e_b).$$

Do not renormalize this pair if $\mu_a \mu_b \geq 2$. FP16/TF32 significant products need at most 22 bits, and BF16 products at most 16, but BF16/TF32 products can lie outside FP32’s range. They neither overflow nor underflow individually at FP32’s limits. This decoder, including subnormal multiplicands, follows the paper-associated MATLAB Tensor Core v0.5 [27].

- 2) **Maximum Exponent Determination:** Select η from the raw product scales ρ_i and the scale of nonzero c . A normal FP32 c contributes its unbiased exponent, while a nonzero subnormal contributes -126 . Ignore zero products and $c = 0$. Apply the alignment floor: -132 on A100 and -133 on Hopper. These floors matter for tiny BF16/TF32 products, including $c = 0$; they do not constrain nonzero FP16 products. If all terms vanish, return zero separately. The maximal normalized value exponent $\alpha = \max_{T_i \neq 0} \lfloor \log_2 |T_i| \rfloor$ is distinct from η .
- 3) **Term Alignment:** With $T_i = a_i b_i$ for $i \leq K$, $T_n = c$, and $q_A = 2^{\eta-F}$, retain

$$u_i = \text{trunc}_{q_A}(T_i), \quad \text{trunc}_q(x) = \text{sgn}(x)q \lfloor |x|/q \rfloor.$$

This truncates magnitudes toward zero and then applies the signs.

- 4) **Accumulation:** Form $A_{\text{acc}} = \sum_i u_i$ exactly, without intermediate normalization. A sufficient software width for a two's-complement accumulation of u_i/q_A is

$$w = F + 3 + \lceil \log_2 n \rceil.$$

Indeed, every $|u_i/q_A| < 2^{F+2}$, so every prefix has magnitude less than $n2^{F+2} \leq 2^{w-1}$. Thus this width does not wrap, in any ordering. It is a conservative software width, not an identification of the reference's W with a signed register width.

- 5) **Final Normalization:** Locate the leading bit of the exact signed accumulator and adjust its scale, including the subnormal-output case. A zero sum has a separate branch.
- 6) **Truncation:** For FP32 output and $|A_{\text{acc}}| \leq \text{maxFinite}_{32}$, retain the FP32 significand toward zero, with gradual underflow. Write this output map as R_θ , so $D = R_\theta(A_{\text{acc}})$.
- 7) **Rounding:** Where the documented FP16-output path normalizes/truncates to FP32 and subsequently rounds to FP16 with nearest-even, retain that ordered sequence [18, Figs. 2, 3, 5]. Our subsequent bounds and scalar correction results concern FP32 output and do not replace that sequence by an assumed one-step rule.

The values T_i suffice to define the ideal sum, but not the Tensor Core output: the factorization fixes raw scales used during alignment. For example, on V100, 1.5×1.5 and 1×2.25 both equal $9/4$, yet with two other products 2^{-23} , unused zero products, and $c = 0$, the outputs are $9/4 + 2^{-22}$ and $9/4$, respectively [18, Sec. 4.1.1].

Unlike traditional two-term scalar addition, all aligned T_i participate in this block accumulation before output conversion. Guard, round, and sticky bits support correctly rounded scalar operations [28]. Tensor Core padding extends the alignment grid, but does not by itself guarantee correct rounding or monotonicity.

III. FORMAL CHARACTERIZATION OF TENSOR CORE ERROR

This section dissects the arithmetic errors from two perspectives: first, by establishing a rigorous upper bound on the deviation from the true ("infinite precision") sum, and second, by analyzing non-monotonicity. Table II includes our glossary of symbols.

A. Bounding the Worst Case Error of Tensor Cores

One of the most fundamental properties of any floating point arithmetic operation is its maximum possible deviation from the true mathematical result. Establishing this worst-case bound provides a guarantee on the quality of the computation. We formalize a conservative bound in Equation 5.

Theorem III.1 (Worst-Case Error of n -Term Addition, ϵ_D). *Use the globally aligned FP32 model in §II-B, with exact no-wrap accumulation and $|A_{\text{acc}}| \leq \text{maxFinite}_{32}$. Define $\epsilon_D =$*

TABLE II: Definitions of key terms and variables used in the proofs.

Symbol	Definition
K, n	Products per globally aligned block; $n = K + 1$
p, F	Padding bits; fractional alignment width $F = 23 + p$
T_i, S	Exact product or c value; $S = \sum_i T_i$
ρ_i, η	Raw product scale; actual alignment exponent
α, γ	Maximum nonzero term value exponent; normal output exponent
q_A, q_D	Alignment quantum; FP32 output quantum
u_i, A_{acc}	Aligned term; exact sum of aligned terms
r_i, r_{out}	Alignment residual; signed output residual
D, D_R, D_{CR}	Hardware output, exact recovered sum, nearest-even result
q_E, τ	Extraction grid; $\log_2(q_E/q_A)$
h_i, H	Coarse retained term; $H = \sum_i h_i$
ϵ_i, ϕ	$T_i - h_i$; $\phi(x) = \text{trunc}_{q_A}(x)$
ϵ_o	Signed correction $D - H$
P, ℓ, L	Scalar precision; common bit weight exponent; absolute coefficient sum
ω, Δ_A	Change in retained products; retained accumulator decrease

$|S - D|$. Let $q_D = 2^{\gamma-23}$ if D is normal with exponent γ , and $q_D = 2^{-149}$ if D is subnormal or zero. Then

$$\epsilon_D < nq_A + q_D. \quad (5)$$

Proof. The computational error arises from two sources: bit loss during initial alignment, and bit loss during normalization and truncation of the accumulated sum. Define the signed residuals $r_i = T_i - u_i$ and $r_{\text{out}} = A_{\text{acc}} - D$. Magnitude truncation gives $|r_i| < q_A$. FP32 output truncation gives $|r_{\text{out}}| < q_D$, including gradual underflow. Since

$$S - D = \sum_i (T_i - u_i) + (A_{\text{acc}} - D),$$

the triangle inequality yields the stated bound. The no-wrap condition ensures that A_{acc} is the exact sum of the retained terms. $\square$

Both contributions are needed, even when raw and normalized product exponents agree. On V100 let four FP16 products equal $(3/2)(5/4) = 15/8$, and let FP32 $c = 1 - 2^{-24}$. For $q = 2^{-23}$, $\eta = 0$ and

$$A_{\text{acc}} = 17/2 - q, \quad D = 17/2 - 8q, \quad S = 17/2 - q/2.$$

Thus $|S - D| = 7.5q$, so a strict $7q$ bound counting only the final loss would be insufficient. Equation 5 accounts for both stages and gives $|S - D| < 13q$ in this case.

An absolute error bound avoids dividing by a potentially tiny output under catastrophic cancellation. The alignment quantum depends on the supplied operand representations and the architecture's exponent policy. The output quantum additionally captures normalization and subnormal boundaries. The bound is sufficient, and is not claimed to be sharp.

B. Preconditions for Observing Non-Monotonicity

While the worst-case error provides an upper bound on the magnitude of error, it does not capture all the nuances of the hardware's behavior. A particular consequence of certain block

summation schemes is non-monotonicity, a counterintuitive behavior which we now formally define.

Definition III.2 (Monotonicity of an Adder). Fix the input format, instruction profile θ , and encoded multiplicands a_i, b_i . The block is monotonic in its FP32 accumulator input if, for all finite $c' < c$ in the domain,

$$\text{val}(\text{TC}_\theta(a, b, c')) \leq \text{val}(\text{TC}_\theta(a, b, c)).$$

A reversed strict output inequality is a witness of non-monotonicity. The comparisons use signed numerical order, not absolute magnitude. Varying an A or B operand instead is a different predicate and must account for the fixed factor's sign and the new raw scale.

Non-monotonicity arises from a specific interplay between a large-magnitude term and a set of smaller-magnitude terms. The large term sets a high alignment exponent that right shifts the smaller terms. If the largest term is then decreased, the alignment exponent may drop, causing previously lost bits of the smaller terms to “flow back” into the adder. A larger exact retained sum need not, however, produce a larger encoded output: final truncation can hide the increase.

Hardware designers include internal padding bits (p), extra bits of precision to catch bits that would otherwise be truncated. The effectiveness of this padding depends on the number of summands, the attainable products, and the output quantization.

Definition III.3 (Flowback for an Accumulator Perturbation). Let q_A, q'_A be the actual alignment quanta for fixed products and accumulator inputs c, c' . Define

$$\omega = \sum_{i=1}^K (\text{trunc}_{q'_A}(T_i) - \text{trunc}_{q_A}(T_i)),$$

$$\Delta_A = \text{trunc}_{q_A}(c) - \text{trunc}_{q'_A}(c').$$

Then $A'_{\text{acc}} = A_{\text{acc}} + \omega - \Delta_A$.

Consequently, the necessary and sufficient encoded-output condition is

$$R_\theta(A_{\text{acc}} + \omega - \Delta_A) > R_\theta(A_{\text{acc}}). \quad (6)$$

Since finite FP32 truncation is monotonic in its real argument, $\omega > \Delta_A$ is necessary. It is sufficient when both accumulated values are exactly representable. The simpler condition $\omega > c - c'$ additionally requires that the two accumulator inputs be retained exactly on their respective grids.

The following theorem gives an explicit family with realizable operand encodings. Its threshold concerns this family; it is not a necessary condition for all possible Tensor Core non-monotonicity.

Theorem III.4 (Hardware Conditions for a Non-Monotonicity Construction). Let $F = 23 + p$, with integer $p \geq 0$, and let $c = 1$ and $c' = 1 - 2^{-24}$ be adjacent FP32 values. Suppose all $K = n - 1$ products equal $t = 2^{-24-p}$, their raw scales are $-24 - p$, and the alignment floor is below -1 . Assume

TABLE III: Non-monotonicity witnesses for the documented FP16/FP32 paths. Fixed $a_i = 2^{-12}$, $b_i = 2^{-12-p}$; decreasing c from 1 to $1 - 2^{-24}$ increases the output. These are exact model calculations.

GPU	n	p	D bits	D' bits
V100	5	0	3f800000	3f800001
A100	9	1	3f800000	3f800001
H100/H200	17	2	3f800000	3f800001

$1 \leq K < 2^{24+p}$. This construction produces $D' > D$ if and only if

$$K \geq 3 \cdot 2^p. \quad (7)$$

Proof. With $c = 1$, the alignment exponent is $\eta = 0$, so $q_A = 2^{-23-p} = 2t$. Every product is truncated to zero, and $D = 1$. With $c' = 1 - 2^{-24}$, its scale is -1 and the raw product scales are smaller. Thus $\eta' = -1$, $q'_A = t$, and all products are retained. The value c' is an integer multiple of t , and

$$A'_{\text{acc}} = 1 + (K - 2^p)t. \quad (8)$$

If this is below 1, the output cannot exceed 1. If it is at least 1, it lies below 2 by the bound on K , and the first larger FP32 value is $1 + 2^{-23} = 1 + 2^{p+1}t$. Truncation therefore produces $D' > 1$ exactly when $K - 2^p \geq 2^{p+1}$, establishing Equation 7. $\square$

Practical Significance: For $p = 0, 1, 2$, the products are realized by normal FP16 operands $a_i = 2^{-12}$ and $b_i = 2^{-12-p}$. Their FP16 encodings are 0c00 and, respectively, 0c00, 0800, or 0400. The two FP32 accumulator encodings are 3f800000 and 3f7fffff. Applying the theorem gives the model-derived witnesses in Table III. The same real powers are representable in BF16 and TF32, but the relevant instruction's K must be used.

These witnesses establish susceptibility of the selected FP16 paths. Failure of this construction on another profile does not prove monotonicity there. In particular, no generation-wide immunity result or sharp universal padding threshold follows. The raw multiplicand decoder, floors, grouping, and attainable inputs are part of any such proof.

C. Extent of Non-Monotonic Perturbation

The maximum output increase and the largest input decrease that can be overcome are distinct optimization problems. We can determine both exactly within the family above.

Theorem III.5 (Range of Non-Monotonic Perturbation and Effect). In the construction of Theorem III.4, replace c' by $c_j = 1 - j2^{-24}$, where $1 \leq j \leq 2^{23}$ is an integer. All such inputs have scale -1 . A non-monotonic increase occurs precisely for

$$1 \leq j \leq J = \min\{2^{23}, \lfloor K/2^p \rfloor - 2\}. \quad (9)$$

If $J \geq 1$, the largest input decrease in this family is $J2^{-24}$, whereas the largest output increase is

$$M = 2^{-23} \left\lfloor \frac{K - 2^p}{2^{p+1}} \right\rfloor. \quad (10)$$

Proof. Each c_j is on the retained grid t , and the products contribute Kt after the scale change. Thus

$$A_j = 1 + (K - j2^p)t.$$

The output exceeds 1 exactly when $K - j2^p \geq 2^{p+1}$, giving Equation 9. Among these witnesses $A_j \in [1, 2)$, so

$$D_j - 1 = 2^{-23} \left\lfloor \frac{K - j2^p}{2^{p+1}} \right\rfloor.$$

This is largest at $j = 1$, while $1 - c_j = j2^{-24}$ is largest at $j = J$. These different extrema give the two formulas. $\square$

For instance, in the hypothetical $K = 5, p = 0$ version of this construction, the maximum input decrease is $3 \cdot 2^{-24}$ and the maximum output increase is $2 \cdot 2^{-23}$. Neither is a universal extremum across all realizable operands. For a general perturbation, Equation 6, rather than $\omega - \Delta$ alone, governs the encoded comparison.

IV. ERROR FREE TRANSFORMATION FOR TENSOR CORES (TC-EFT): THEORETICAL BASICS

Our work extends the idea of obtaining (R, e) reviewed earlier (§II-A1) to the multi-term summations performed by Tensor Cores. Analogously, TC-EFT returns a collection of correction components whose signed sum, together with the hardware output, equals the exact result. Such an expansion need not fit in a single machine value.

The goal now is to capture errors that account for the bits lost at the different stages of alignment and output conversion. There are three separate obligations: recover the correct mathematical components, construct those components exactly in software, and perform a single final nearest-even rounding of their exact sum. A small exponent span is useful only after the components themselves are correct.

Much like in standard EFT, we need information in addition to D . Here the original encoded A, B, c supply both the exact values T_i and the raw scales that determine η . Product values alone are insufficient, as the factorization example in §II-B demonstrates. A hardware interface exposing raw products, or their values together with sufficient scale metadata, would also supply this information.

A quick attempt would be to extract low bits per term and add their sum to D . This requires care: some extracted bits may already have participated in the Tensor Core accumulation. We retain the overlap formulation, but derive its window from the actual alignment and extraction grids. Lemmas IV.1–IV.4 identify the extracted components and their signed correction. Theorem IV.5 establishes the recovery identity; Theorems IV.8–IV.9 and Corollary IV.11 give sufficient scalar-consolidation conditions. Algorithm 1 specifies an exact reference implementation. Computing the coarse retained sum there requires additional integer work; a cheaper extractor must prove the same identity.

A. Compute Bits Lost Based on Result Exponent

Recall from §II-B that the hardware aligns the raw terms to η , with quantum $q_A = 2^{\eta-(23+p)}$. Alignment truncates individual terms; final normalization and output truncation can lose bits of their accumulated sum. For a normal output let γ be its exponent and $\delta = \gamma - \eta$. When D is zero or subnormal, use its output quantum q_D directly, without taking a logarithm of zero.

Choose the coarse extraction grid

$$q_E = \max(q_A, q_D), \quad q_E = 2^{\psi-23}, \quad E = 2^\psi. \quad (11)$$

The powers q_E, E are mathematical scales; they need not be machine FP32 values. All subsequent results also hold for another power-of-two $q_E \geq q_A$. This choice is independent of the largest normalized product exponent. It does not assert that adding E in scalar FP32 extracts the required bits.

Lemma IV.1 (Isolation of Lost Bits). *For an exact dyadic term T_i , let $h_i = \text{trunc}_{q_E}(T_i)$. Its signed low component is*

$$\epsilon_i = T_i - h_i = \text{sgn}(T_i)(|T_i| - q_E \lfloor |T_i|/q_E \rfloor). \quad (12)$$

Then $T_i = h_i + \epsilon_i$ and $|\epsilon_i| < q_E$ in exact arithmetic.

Proof. Euclidean quotient and remainder of the nonnegative dyadic $|T_i|$ at grid q_E give a remainder in $[0, q_E)$. Restoring the original sign gives the stated decomposition. Both parts have the sign of T_i , unless zero. This calculation can be implemented using integer quotient and remainder after expressing the term and grid at a common binary scale. $\square$

The ϵ_i terms capture the low part of each term on the chosen coarse grid. They are not all alignment losses: the hardware can retain some of their bits before the output conversion. Adding all ϵ_i to D without accounting for the retained contribution can therefore double-count it.

Lemma IV.2 (Overlap Window Width). *The number of bit positions between the two grids is*

$$\tau = \log_2(q_E/q_A) = \psi - \eta + p \geq 0. \quad (13)$$

For Equation 11, $\tau = \max(0, \log_2(q_D/q_A))$. For normal D this is $\max(0, \delta + p)$.

Proof. The potentially retained weights in the extracted component are $q_A, 2q_A, \dots, 2^{\tau-1}q_A$, since $|\epsilon_i| < q_E = 2^\tau q_A$. There are τ such positions. Substitution of the two grid exponents gives Equation 13. $\square$

Lemma IV.3 (Isolation of Overlapping Bits). *Define the retained part of an extracted component by*

$$\phi(\epsilon_i) = \text{trunc}_{q_A}(\epsilon_i). \quad (14)$$

Then $u_i = h_i + \phi(\epsilon_i)$, and the actual alignment residual is $r_i = \epsilon_i - \phi(\epsilon_i)$.

Proof. The coarse retained magnitude $|h_i|$ is an integer multiple of q_A because q_E/q_A is an integer power of two. Since h_i and ϵ_i have the same sign, magnitude truncation splits into the retained high part and the truncated remainder. Restoring

that common sign proves the first equality. Subtracting from $T_i = h_i + \epsilon_i$ proves the second. This argument is per term; it does not commute signed truncation through a mixed-sign sum. $\square$

The collective sum of the $\phi(\epsilon_i)$ can contribute to D , but the output map also depends on the signed high sum. A carry-only description is insufficient under cancellation. For example, for grid $q = 1$, $\text{trunc}_1(1 - 1/2) = 0$, while $1 + \text{trunc}_1(-1/2) = 1$.

Lemma IV.4 (Overlap Correction for Double-Counted Contributions). *Let $H = \mathcal{E}_i h_i$ and $V = \mathcal{E}_i \phi(\epsilon_i)$. With $r_{\text{out}} = A_{\text{acc}} - D$, the signed overlap correction is*

$$\epsilon_o = D - H = V - r_{\text{out}}. \quad (15)$$

Proof. Lemma IV.3 gives $A_{\text{acc}} = H + V$. Subtracting H from $D = A_{\text{acc}} - r_{\text{out}}$ establishes the identity. All sums and differences here are exact. In particular, this statement does not establish representability of ϵ_o in FP32 or a way to compute it from unsigned overlap carries alone. $\square$

With these components defined, we can formulate the recovery theorem.

Theorem IV.5 (Exact Sum Recovery). *For the model in §II-B, with finite D and exact construction of the components, the exact sum is recovered as*

$$D_R = S = D - \epsilon_o + \sum_{i=1}^n \epsilon_i. \quad (16)$$

Equivalently, using the stage residuals,

$$S = D + r_{\text{out}} + \sum_{i=1}^n r_i. \quad (17)$$

Proof. By Lemma IV.1, $S = H + \mathcal{E}_i \epsilon_i$, and by Lemma IV.4, $H = D - \epsilon_o$. This proves Equation 16. Independently, $\mathcal{E}_i r_i = S - A_{\text{acc}}$ and $r_{\text{out}} = A_{\text{acc}} - D$ telescope to Equation 17. The proof permits arbitrary signs and exact dyadic products outside FP32's range. $\square$

For $q_E = q_A$, $H = A_{\text{acc}}$ and $\epsilon_o = -r_{\text{out}}$, so the overlap form reduces to the stage-residual form. For a coarser grid, computing H exactly is a sound reference construction but may redo substantial accumulation work. Recovering ϵ_o more cheaply from a small overlap window requires a further proof, including signed cancellation and output conversion.

a) *Why raw scales matter:* On V100, put $u = 1 + 2^{-10}$ and choose

$$a = (3/2, u, u, 1), \quad b = (3/2, u/8, u/8, 1/8), \quad c = 1/8.$$

The raw product scales are $(0, -3, -3, -3)$, so $\eta = 0$ and $q_A = 2^{-23}$. Every term is on that grid. Their exact sum is

$$S = 11/4 + 2^{-11} + 2^{-22} = \frac{11536385}{4194304},$$

with FP32 encoding 40300801. Hence all stage residuals vanish. For the coarse choice in Equation 11, $q_E = 2^{-22}$, the two small product remainders are each 2^{-23} , and $\epsilon_o = 2^{-22}$; they cancel exactly in the recovery formula. Treating the

largest product's normalized exponent $\alpha = 1$ as the hardware alignment exponent instead misses this retained contribution and produces 40300802. Exact summation of incorrect components cannot repair that error.

B. TC-EFT: Correct Rounding

In Equation 16, D is the answer computed by the Tensor Core, onto which we must incorporate $\epsilon_o, \epsilon_1, \dots, \epsilon_n$. To compute D_{CR} on scalar hardware, we first need an exact $D_o = D - \epsilon_o = H$, then an exact sum $\epsilon_{\text{tot}} = \sum_i \epsilon_i$, followed by a single nearest-even rounding of $D_o + \epsilon_{\text{tot}}$ to the output format.

Neither exactness of the subtraction nor exactness of a naive summation is automatic. We give sufficient conditions in terms of the actual components' binary support and range. These retain the useful scalar exactness argument while distinguishing the FP32 hardware-output precision from the software correction precision P .

Definition IV.6 (Naive Summation). *For floating-point values $T_1, \dots, T_n$ in a fixed binary format, a naive summation in a chosen ordering is*

$$S_1 = T_1, \quad S_k = S_{k-1} \oplus T_k \quad (k = 2, \dots, n),$$

where $\oplus$ is a correctly rounded scalar addition with gradual underflow. An assertion valid in every ordering refers to every permutation of these inputs.

For a naive summation to be exact at every step, each intermediate two-term sum must be representable in the scalar format. This ideal is captured in the following lemma.

Lemma IV.7 (Condition for Exact Two-Term Sum). *If a, b are finite values in the scalar format and the exact real sum $a + b$ is also a finite representable value, a correctly rounded scalar addition returns $a + b$ exactly.*

Proof. Correct rounding fixes a representable exact result: that result is at distance zero from itself and is therefore selected. This follows from the operation's specification; it does not require every bit shifted during a hardware alignment to be zero. Cancellation, for example $1 - (1 - 2^{-24}) = 2^{-24}$ in FP32, can produce an exact result despite a large shift. $\square$

Significance: A common binary grid and a bound on the sum of absolute coefficients can ensure that every intermediate sum fits, including mixed-sign orderings and subnormal results.

Theorem IV.8 (Condition for Exact N-Term Sum). *Let the scalar format have precision P , minimum positive subnormal $q_{\min, P}$, and largest finite magnitude M_P . Suppose $T_i = z_i 2^\ell$, with $z_i, \ell \in \mathbb{Z}$ and*

$$2^\ell \geq q_{\min, P}, \quad L = \sum_i |z_i| < 2^P, \quad L 2^\ell \leq M_P. \quad (18)$$

Then every naive summation of these terms is exact.

Proof. For any ordering and any prefix, its exact coefficient z is an integer satisfying $|z| \leq L < 2^P$. Thus it needs at most

P significant binary digits. Its value $z2^\ell$ lies within the finite range by the last inequality. If subnormal, it lies on the format's subnormal grid by the first inequality. Consequently every input and every prefix sum is representable, including zero. Induction over prefixes, using Lemma IV.7, proves exactness of each scalar addition. $\square$

A convenient stronger sufficient bound can be expressed through a bit span. For integers b, ℓ , if $|T_i| < 2^{b+1}$ and every term is on grid 2^ℓ , then $\sum_i |T_i| < n2^{b+1}$. Hence $b - \ell + 1 + \lceil \log_2 n \rceil \leq P$, together with the range and grid requirements, suffices. The bound applies to every prefix because it bounds the sum of absolute input magnitudes, not merely the final possibly cancelled result.

Theorem IV.9 (Precondition for Correct Rounding). *Construct the actual ϵ_i of Lemma IV.1 exactly. Choose a power-of-two grid 2^ℓ on which all these components lie, and put $z_i = \epsilon_i 2^{-\ell}$. If*

$$2^\ell \geq q_{\min, P}, \quad \sum_i |z_i| < 2^P, \quad \sum_i |\epsilon_i| \leq M_P, \quad (19)$$

then their naive scalar sum is exact. One sufficient input-based bound, for $q_E = 2^b$ and $\ell \leq b$, is

$$n(2^{b-\ell} - 1) < 2^P, \quad (20)$$

provided T_i and q_E are on grid 2^ℓ and the same range and minimum-grid requirements hold.

Proof. The first statement is Theorem IV.8 applied to the constructed residuals. For the second, ϵ_i is an integer multiple of 2^ℓ and $|\epsilon_i| < q_E = 2^b$. Therefore $|z_i| \leq 2^{b-\ell} - 1$. Summing this bound establishes the coefficient condition. If all $\epsilon_i = 0$, their sum is zero and no nonzero-support exponent is needed. $\square$

This condition includes the c remainder. When r_i, r_{out} are consolidated directly instead, Equation 18 must hold for all $n+1$ of those components. The hardware alignment precision remains $F = 23 + p$ even when software uses $P = 53$. Increasing software precision does not change the extractor or the hardware grid.

Lemma IV.10 (Exactness of Overlap Correction). *If D , ϵ_o , and $H = D - \epsilon_o$ are finite representable values in the correction format, then $D \ominus \epsilon_o = H$ exactly under correctly rounded scalar subtraction. Otherwise H can be retained as an exact integer dyadic, but this single scalar subtraction is not justified.*

Proof. Apply Lemma IV.7 to D and $-\epsilon_o$. Representability is a hypothesis, not a consequence of the word “overlap.” $\square$

Corollary IV.11 (Application to EFT). Assume $|S| \leq \text{maxFinite}_{32}$, the exact recovery identity, exact construction of all components, Theorem IV.9, and Lemma IV.10. Then a single correctly rounded operation on the exact sum $H + \epsilon_{\text{tot}}$ produces

$$D_{CR} = \text{RN}_{\text{even}, 32}(S).$$

TABLE IV: Constants for the exact scalar-consolidation predicate. Also require the absolute sum to be within the format's finite range and the overlap subtraction to satisfy Lemma IV.10.

Format	P	$q_{\min, P}$	Coefficient bound
FP32	24	2^{-149}	$L < 2^{24}$
FP64	53	2^{-1074}	$L < 2^{53}$

In particular, if the correction format is FP32, this operation is one FP32 nearest-even addition $H \oplus_n \epsilon_{\text{tot}}$.

Proof. The subtraction yields H exactly, and the residual accumulation yields ϵ_{tot} exactly. Their real sum is S by Theorem IV.5. Rounding that sum once to the target format is therefore the required result, including ties and subnormal boundaries. $\square$

Practical Implications of Our Approach: The conditions can be checked from the actual dyadic components. Table IV gives the format constants used in those checks; it does not infer a universal operand-exponent interval. Product scales, c , and cancellation all affect the components. In particular, the normalized-term exponent-span condition cannot validate a recovery formula that omits already-retained bits, as the witness in §IV-A shows.

If FP64 is used to consolidate components for an FP32 result, an FP64 rounded addition followed by FP32 conversion can double-round. Use an exact dyadic sum rounded directly to FP32, or a separately proved final-rounding procedure. If the scalar predicate fails, an integer accumulator or a verified expansion can preserve the components exactly. Such a fallback preserves the recovery contract but has a different cost from the scalar path.

V. EVALUATION

We evaluate the recovery procedure against an independent exact oracle. We distinguish the initial GPU experiments from validation of the raw-product model and the reference algorithm specified here. The three questions are whether (1) the chosen model captures the actual instruction, (2) an implementation realizes the model and recovery equations, and (3) the recovered result matches the correctly rounded exact dot product. Agreement on one of these questions does not establish the others.

A. Implementation

The initial TC-EFT prototype used approximately 500 lines of CUDA and requested Tensor Core MMA operations via inline PTX. Given matrices A, B, C , it obtained D , then extracted correction terms through bit manipulation. The original inputs and D remain a sufficient interface for recovery. Since each cell is independent within a specified block, cells may be corrected concurrently across threads.

Algorithm 1 gives the reference procedure for the raw-product semantics. Its accompanying implementation uses exact integer dyadics and explicitly reconstructs the raw significand and scale from operand encodings. It preserves the

recovery identity without assuming that every component fits FP32. This reference implementation has not been substituted into the original CUDA experiments; those experiments do not validate its GPU cost or the earlier extractor on the corrected domain.

a) Bit Manipulations: For each multiplicand, extract the sign, exponent field, and fraction in its input format. Restore the implicit bit only for normal inputs. For a subnormal, retain the explicit fraction and use the minimum normal exponent. Multiply the integer significands exactly and add the operand scales; preserve the product’s second integer bit. Decode c separately as FP32. An encoded FP32 multiplication is not an adequate reconstruction for arbitrary BF16/TF32 products because its range is too narrow.

For an integer dyadic $T_i = \sigma_i M_i 2^{\ell_i}$, extraction at $q_E = 2^b$ uses quotient and remainder of M_i at the appropriate power of two. If $b \leq \ell_i$, the remainder is zero. Otherwise put $s = b - \ell_i$; for s at least the bit length of M_i , the whole magnitude is the remainder; for smaller s , use $M_i \bmod 2^s$. This avoids undefined fixed-width masks such as shifting a 32-bit word by 32 or more. The same construction at q_A yields u_i . Apply signs after magnitude truncation.

Accumulate H/q_E in a signed integer representation with a proved no-wrap bound, or in arbitrary-precision integers. Compute $\epsilon_o = D - H$ exactly, at a common binary scale. Reconstruct scalar values only after checking their support, finite range, and subnormal encoding. Signed overlap terms must not be treated as an unchecked unsigned sum with carry masking: magnitude truncation and two’s-complement arithmetic differ for negative values.

The reference operation ledger appears in Table V. It includes the retained-sum reconstruction, which an inexpensive optimized extractor would need to avoid or accelerate. In the conditional FP32 scalar path, once all components are available exactly, summing n residuals uses $n - 1$ additions, overlap subtraction uses one, and the final nearest-even addition uses one: $n + 1$ scalar operations. Recomputing $K = n - 1$ FP16 products using exact FP32 multiplications would add $n - 1$ operations, giving $2n$, but this count excludes the necessary integer work and does not cover arbitrary-range BF16/TF32 reconstruction. Providing raw products removes product reconstruction, not the subtraction and final rounding. It therefore does not reduce this ledger to $n - 1$ operations overall.

The scalar branch replaces the final exact operations with the operations justified in Corollary IV.11. It additionally incurs $O(n)$ component-representability, support, and range checks, whose integer cost depends on operand width. Counts of scalar floating-point, integer, and Tensor Core instructions are kept separate; no performance advantage of this reference construction is claimed.

B. Ground Truth Oracle

We require a reference value for D_{CR} that is correct by construction. The initial experiments used GNU MPFR [29] at 1024 bits on a CPU, followed by FP32 nearest-even

TABLE V: Per-cell reference recovery work, excluding the initial Tensor Core invocation. Integer additions at differing dyadic scales include the required shifts. This is an operation ledger, not a GPU timing result.

Operation	Count/type
Decode multiplicands, c , and D	$2K + 2$ bit decodes
Select η and extraction grid	$O(n)$ scale comparisons
Reconstruct raw products	K integer multiplies
Construct h_i, ϵ_i	n quotient/remainder extractions
Construct H	$n - 1$ signed integer additions
Construct $\epsilon_o = D - H$	1 exact dyadic subtraction
Accumulate ϵ_i	$n - 1$ exact additions
Form $D - \epsilon_o$, then add residual sum	2 exact additions/subtractions
Round to FP32	1 integer nearest-even conversion

rounding. For the model checks here, an independent path instead decodes the original operand bits, multiplies their exact rational values, adds c exactly, and selects the nearest FP32 neighbor, breaking ties by the low encoding bit. It does not use D , the alignment exponent, extracted corrections, or the recovery algorithm.

The precision requirement is determined by raw-product range, not only FP32 term range. Every finite TF32 value is on grid 2^{-136} and has magnitude below 2^{128} , so every product is on grid 2^{-272} and has magnitude below 2^{256} . For $K \leq 16$ and finite FP32 c , the absolute sum is below 2^{261} , giving a conservative 533-bit exact accumulator span. FP16 and BF16 require no wider span. Thus 1024-bit MPFR is ample for these blocks if its exponent range is configured to include this domain and inexact/overflow flags are checked. The integer oracle avoids dependence on an MPFR precision or exponent-range setting.

C. Experimental Setup

The initial hardware experiment used 100 sets of randomly generated inputs at the granularity of one matrix cell, evaluated on an NVIDIA Tesla V100. For ease of exposition, matrices contained values in one row and column. There were nine inputs per experiment: four values in A , four in B , and one in C .

a) Hardware: V100’s FP16/FP32 path accumulates $n = 5$ terms with $p = 0$. The historical observation applies to this prototype and device. Hardware evaluation of the reference algorithm on V100, and evaluation on the other selected profiles, remain future work. Porting requires the full profile in Table I, including instruction grouping and raw-scale handling.

b) Inputs: The initial generator produced 900 values by sampling a sign, a 23-bit fraction, and an exponent in the intended interval $[0, 9]$. The actual MMA operands are values after conversion to the instruction’s input format; a generator’s FP32 values alone do not specify them. Validation of the scalar precondition must use those post-conversion bits, form the raw products, include c , and check Equation 19 and the overlap-

subtraction hypotheses. An exponent restriction alone does not validate the previous normalized-product extractor.

The exact regression suite records operand encodings for 59 named cases, including the recovery and error-bound witnesses, equal-value/different-factorization pairs, the cancellation example below, subnormal multiplicands, zero blocks, signed cancellation, alignment floors, and products outside FP32’s range. It also checks 800 deterministic blocks across eight profile/format combinations (seed 20260905), 49,005 synthetic perturbations of the family in §III-C, and 100 rounding cases at ties and normal/subnormal boundaries.

A further 1,600 finite blocks sample all finite exponent fields and opposite-product cancellation (seed 20260906). An independent integer-shift oracle checks alignment and truncation, with separate IEEE decoding for nearest-neighbor rounding. Boundary and composition tests and excluded-domain counts accompany the source. These are software calculations, not GPU measurements. Because $\epsilon_o = D - H$ makes recovery an identity, recovery tests alone cannot validate alignment.

D. Results

a) *TC-EFT Correctness*: The initial CUDA prototype reported a bitwise match with its MPFR comparison in all 100 V100 experiments. This observation is retained as a limited historical test result. It does not establish universal agreement with the raw-product model or correctness of the earlier overlap extractor, which fails on the explicit witness in §IV-A.

All of the exact regressions described above pass for the reference implementation: the two recovery identities agree with the independently accumulated ideal sum, direct nearest-even conversion matches the neighbor-search oracle, and the finite-output error bound holds. Whenever the scalar residual-support predicate holds, tested scalar summation orderings are exact. The general scalar guarantee is supplied by the proofs, rather than by these finite tests. No revised CUDA execution or SMT proof result is asserted by this validation.

b) *Worked Example.*: We retain the cancellation example, with the five exact summands:

- 1) $T_1 = -2^{14} \times 1.10010111101010110111000$
- 2) $T_2 = 2^{11} \times 1.11110011101111100010000$
- 3) $T_3 = 2^5 \times 1.10010000111100111100000$
- 4) $T_4 = 2^{14} \times 1.01100010111101100000000$
- 5) $T_5 = 2^9 \times 1.00101101100111111100101.$

A compatible reconstruction of its FP16 operands is

$$\begin{aligned} a_{\text{bits}} &= (\text{d83d}, 5061, 444\text{c}, 5810), \\ b_{\text{bits}} &= (5\text{a03}, 5722, 49\text{d5}, 5976), \\ c_{\text{bits}} &= 4416\text{cfe5}. \end{aligned}$$

These factors reproduce the printed product values; they are a reconstructed witness, not a recovered record of the original GPU operands. We evaluate one V100 FP16/FP32 four-product block ($K = 4$).

The raw scales are $(14, 11, 5, 14)$, so $\eta = 14$ and $q_A = 2^{-9}$. The aligned integer coefficients are

$$(u_i/q_A)_{i=1}^5 = (-13358520, 2046946, 25660, 11631360, 308863).$$

Their exact sum is 654309, yielding

$$A_{\text{acc}} = D = \frac{654309}{512}, \quad D_{\text{bits}} = 449\text{f}\text{be}50.$$

This is the output reported for the original cancellation example. Its normalized exponent is $\gamma = 10$, so $\delta = -4$ and $q_D = 2^{-13}$. With $q_E = q_A$, the overlap window is empty and $\epsilon_o = 0$. The only nonzero components are

$$\epsilon_3 = \frac{15}{8192}, \quad \epsilon_5 = \frac{5}{16384}, \quad \epsilon_{\text{tot}} = \frac{35}{16384}.$$

The exact sum is $S = 20937923/16384$. It is halfway between adjacent FP32 values, and nearest-even selects $D_{CR} = 449\text{f}\text{be}62$, agreeing with the original reported corrected output. The scalar predicate holds on grid 2^{-14} with $L = 35$, and $H = D$ is representable, so the final FP32 addition is justified by Corollary IV.11. Cancellation made bits small relative to 2^{14} significant in the 2^{10} -scale result. Changing the factors can change the alignment, so the encoded reconstruction is part of this trace.

VI. RELATED WORK

We organize related work under three headings. (1) First, we review studies focused on understanding the precise properties of Tensor Cores. There is a growing population of researchers who have taken active interest in this topic, given the importance of moving Tensor Cores into the arena of *reliable* and *efficient* matrix accelerators. This is timely, given the growing need to safely accelerate HPC and ML workloads in this post-Dennard-scaling era. (2) Next, we discuss broader, more applied work in building Tensor Core based solutions. This is an area where solutions pertaining to recovering ground-truth answers will become most relevant. (3) Finally, we survey work in the important area of Error-Free Transformations, including many of their applications.

Characterizing Tensor-Core Arithmetic: The first step towards the reliable use of these accelerators is understanding their precise properties. This has been pursued through increasingly rigorous methods. Early work by Fasi et al. [16] and Hickmann and Bradford [30] used carefully designed, manual tests to probe the hardware and reverse-engineer its properties. These pioneering efforts provided the first public insights into the non-IEEE compliant nature of Tensor Cores, identifying properties like internal precision and rounding modes.

Li et al. [13] advanced this approach by creating a systematic suite of “feature-targeted” tests. These tests were designed to quickly and unambiguously identify differences in key numerical features, including subnormal number support, the number of extra precision bits, rounding modes, and block FMA width across various NVIDIA and AMD accelerators. This work showed that different architectures of Tensor Cores

can produce vastly different results from the same matrix multiplication. This was further built upon by Valpey et al. [17], which employed formal methods to create an executable model of three generations of Tensor Cores. Instead of designing tests by hand, an SMT solver was used to automatically generate discriminating inputs that could prove or disprove hypotheses about the hardware’s internal mechanics. This approach not only confirmed many previous findings but also made several crucial corrections.

Khattak and Mikaitis [18] refine these numerical models using targeted and randomized comparisons with hardware. Their v4 model retains raw significant products with exponent sums, identifies lower bounds on the alignment exponent for tiny BF16/TF32 products, and distinguishes zero from nonzero subnormal accumulators. In particular, the FP32-product representation used by Algorithm 1 of the three-generations model [17] does not retain all alignment metadata required by this refinement. The earlier SMT methodology remains useful, but a proof over that representation is not automatically a proof about the revised instruction model. We use the v4 paper and its associated MATLAB Tensor Core v0.5 [27] as the numerical specification here.

Tensor-Core-based Software Libraries: Given the complex and non-standard behavior of these accelerators, researchers have developed specialized algorithms to leverage their speed for high-precision computations. Mukunoki et al. [31] applies the Ozaki scheme, which decomposes high-precision matrices into several exact, lower-precision components. The components are constructed specifically such that their subsequent matrix products can be computed without any rounding error in the FP32 precision of a Tensor Core’s adder. These components are then multiplied on hardware, and the results are summed to achieve accuracy equivalent to standard DGEMM (FP64).

Fasi et al. [32] uses a similar method termed multi-word arithmetic, where a higher-precision number is represented as the unevaluated sum of multiple lower-precision “words”. For their “double-fp16” approach, an FP32 matrix A is approximated as the sum of two FP16 matrices, $A \approx A_1 + A_2$. The first word, A_1 , is the result of rounding A to FP16, while the second word, A_2 , captures the rounding error from that first step. The matrix product uses Tensor Core products of these words (e.g., $A_1B_1 + A_1B_2 + A_2B_1$) and FP32 summation. Their blocked summation algorithm (FABsum) improves the performance–accuracy tradeoff without guaranteeing bitwise FP32 equivalence. Double-fp16 can be less accurate than FP32 despite the same worst-case error bound.

These works differ from our own as they are high-level frameworks that treat the Tensor Core as a black-box accelerator to compute an entire matrix product. Their goal is to approximate or achieve equivalence with a standard precision format (FP32 or FP64). Our algorithm operates on one globally aligned numerical block. Our work focuses on a crucial aspect of these operations—namely multi-term addition—to recover correctly rounded results under the stated

model and exact-consolidation hypotheses. This allows us to focus on recovering correctly rounded block results under a specified instruction model, with its raw-product and alignment semantics made explicit. A tile or instruction containing several such blocks requires composition of their residuals.

Related Work on Error-Free Transformations (EFT): The area of Error-Free Transformations has a rich history beginning with the work of Dekker [21] and Knuth [22] who introduced seminal ideas in terms of recovering floating-point computational error using floating-point operations themselves. EFTs have been used to accompany numerically sensitive algorithms to correct for errors or to provide trustworthy baseline results [33, 34]. Classic algorithms like Dekker’s Fast2Sum [21] are designed to recover the exact rounding error from a standard two-term floating-point addition. Kernerup et al. [35] have studied the minimality of these basic blocks, proving that 2Sum is optimal in terms of operation count and dependency graph depth for algorithms without comparisons. Jeannerod and Zimmermann [24] also recently revisited Fast2Sum, providing new, less restrictive conditions for its exactness and tighter error bounds for cases where it is not a perfect EFT. Our work transitions these core ideas from a two-term sum ($a + b$) to the n -term summation performed within a specified Tensor Core block.

The work in Chowdhary and Nagarakatte [23] uses EFTs to build a floating-point sanitizer—essentially a baseline result against which computed results can be compared. Our work on TC-EFT is similarly motivated—intending to (eventually) support a verification framework for algorithms being migrated to use Tensor Cores. Zhang and Aiken [34] introduced the concept of “Floating-Point Accumulation Networks” (FPANs) to describe branch free algorithms for higher precision built from TwoSum blocks. They present an automatic verification procedure to prove error bounds for these networks. Our work also contributes to this space by formally characterizing a hardware alternative to FPANs—the Tensor Core’s internal adder—and providing its corresponding EFT.

Beyond two term sums, significant work has focused on computing correctly rounded sums of multiple terms too. Boldo and Melquiond [36] introduced algorithms using “rounding to odd” mode for intermediate steps to compute the correctly rounded sum of three floating-point numbers. Boldo et al. [37] built on this work and described a technique for correctly rounding the dot product operation in fixed point arithmetic.

VII. PSEUDOCODE OF TC-EFT, RECOMMENDATIONS FOR GPU DESIGN

In this work, we have derived a method for recovering the correctly rounded result of a globally aligned Tensor Core block under the stated numerical model. Algorithm 1 composes the equations from the preceding sections. It makes exact integer construction explicit and invokes scalar consolidation only when its hypotheses hold.

TC-EFT requires these salient steps:

Algorithm 1 Reference TC-EFT for one globally aligned FP32-output block with $|S| \leq \text{maxFinite}_{32}$. Inputs are operand encodings, profile θ , and its finite hardware result D . All unmarked arithmetic is exact signed integer-dyadic arithmetic.

```

1: procedure TCEFT( $a, b, c, D, \theta$ )
2:    $K \leftarrow \theta.K$ ;  $n \leftarrow K + 1$ 
3:   for  $i = 1, \dots, K$  do
4:     Decode  $a_i, b_i$  in their input format
5:      $(T_i, \rho_i) \leftarrow$  exact raw product and scale
6:      $T_n \leftarrow \text{val}(c)$ ; decode its FP32 scale
7:     if all  $T_i = 0$  then
8:       return  $+0$ 
9:      $\eta \leftarrow$  raw-scale maximum with  $c/\text{floor}$  policy
10:     $q_A \leftarrow 2^{\eta - (23 + \theta.p)}$ 
11:     $q_D \leftarrow$  FP32 output quantum, with zero branch
12:     $q_E \leftarrow \max(q_A, q_D)$ 
13:    for  $i = 1, \dots, n$  do
14:       $h_i \leftarrow \text{trunc}_{q_E}(T_i)$ 
15:       $\epsilon_i \leftarrow T_i - h_i$ 
16:     $H \leftarrow \text{ExactSum}(h_1, \dots, h_n)$ 
17:     $\epsilon_o \leftarrow \text{val}(D) - H$ 
18:    if FP32 hypotheses of Corollary IV.11 hold then
19:       $\epsilon_{\text{tot}} \leftarrow$  FP32 naive sum of  $\epsilon_i$ 
20:       $D_o \leftarrow D \ominus \epsilon_o$ 
21:      return  $D_o \oplus_n \epsilon_{\text{tot}}$ 
22:    else
23:       $D_R \leftarrow D - \epsilon_o + \text{ExactSum}(\epsilon_i)$ 
24:      return  $\text{RN}_{\text{even}, 32}(D_R)$ 

```

- 1) Decode A, B, c and form raw products, retaining the operand exponent sums (line 5).
- 2) Select the actual alignment grid and an extraction grid, then construct the n low components ϵ_i (line 15).
- 3) Construct the signed retained sum H exactly and obtain $\epsilon_o = D - H$ (line 17).
- 4) Check the scalar support, range, and representability conditions before accumulating the components (line 19).
- 5) Form $D - \epsilon_o$ exactly (line 20), or retain the exact dyadic expansion if the scalar conditions fail.
- 6) Round the exact recovered sum once to FP32 with ties-to-even (line 21 for the scalar path).

Unfortunately, this reference construction requires redoing work already done inside the Tensor Core: constructing products and a retained sum. This cost can be mitigated if future Tensor Core-like hardware provides a mechanism for reusing computed raw AB terms. The interface must retain the scale metadata as well as the product values. Access to A_{acc} itself would further expose r_{out} through its difference from D , leaving the per-term alignment residuals to be recovered. Merely exposing normalized FP32 products does not supply the full information, particularly for BF16/TF32 products outside FP32’s range.

A key implication centers on the internal multi-term adder. Currently, it is accessed through the fixed matrix multiply-accumulate operation. If manufacturers exposed it as an independent callable primitive with a documented input representation and alignment policy, it could support residual accumulation. To replace the exact residual sum, that primitive would need to preserve the components’ common grid, provide enough signed carry capacity, and avoid an inexact intermediate output conversion. The scalar support theorem alone does not imply these properties for a Tensor Core adder. Thus the proposed four-invocation implementation remains conditional on an additional primitive specification and exactness proof.

Such hardware could offer a useful alternative to error accumulation through Kahan summation [38] or EFT-composition methods such as Floating-Point Accumulation Networks [34]. Their operation counts, accuracy guarantees, and costs differ; the present reference construction does not establish a speed advantage over them. An efficient implementation with the same recovery invariant would be valuable for large-scale scientific simulations and verification of matrix algorithms.

Another key insight pertains to non-monotonicity avoidance. Table III gives concrete witnesses for selected profiles, and Theorem III.4 explains the role of padding for that family. Adding one padding bit prevents these particular FP16 witnesses on the listed block widths. Establishing immunity for all inputs would require a separate proof over realizable products, accumulator values, alignment floors, and encoded outputs. It cannot be inferred from the failure of one construction.

These design questions can help guide the transition of Tensor Cores from domain-specific accelerators toward general numerical computing units with explicit accuracy contracts. The required hardware resources and performance effects remain to be measured.

VIII. CONCLUSIONS AND FUTURE WORK

We presented a formal characterization of numerical error for specified globally aligned FP32-output Tensor Core models. Specifically, we bound the discrepancy between the computed result D and the exact real dot product by accounting for both alignment loss and output truncation. We demonstrate connections between the cause of this discrepancy and non-monotonic behavior, with realizable operand-level witnesses and separate formulas for input perturbations and output increases in a defined family.

Using this characterization, we express TC-EFT through exact stage residuals and an equivalent signed overlap decomposition. Correct rounding follows when these components are constructed exactly and consolidated under the stated scalar support and range conditions, or when their exact dyadic sum is rounded directly. The independent reference implementation checks this contract, including the original cancellation example and cases that distinguish raw-product alignment from a normalized-product model. The earlier V100 observations remain useful experimental evidence within their tested scope; revised GPU implementation and validation are still required for an efficient Tensor Core sanitizer.

The insights discussed in this work are essential for the continued evolution of hardware accelerators. Beyond supporting software accuracy checks, a formal characterization of the internal accumulation scheme can inform future hardware design. Specifically, this work motivates documented raw product and accumulator interfaces and a standalone multi-term adder whose representation, range, and rounding behavior can be reasoned about directly.

If the multi-term adder situated immediately after the multiplication units were available separately, it could reduce the cost of correction under a suitable exactness contract. We advocate investigating that interface. The claimed low-cost overlap extractor, universal padding immunity, and comparative performance benefits require further proofs or measurements; the reference recovery identity does not establish them.

Finally, our work focuses on one globally aligned block, which need not equal an entire PTX instruction, matrix tile, or GEMM operation. For a serial schedule with hardware output d_j used as the next accumulator, let e_j be the exact local residual satisfying $d_{j-1} + P_j = d_j + e_j$, where P_j is the exact product sum of block j . Telescoping gives

$$c_0 + \sum_j P_j = d_m + \sum_j e_j.$$

This is an exact composition invariant, provided all local residuals are retained. Efficiently representing and consolidating that ledger, or accounting for additional external scalar additions in a different schedule, remains future work. Rounding each block correctly is not by itself a proof of correct rounding for a larger matrix product.

Data-Availability Statement: The editable source is accompanied by an exact dyadic reference model, an independent exact-sum and nearest-neighbor oracle, the regression inputs, and machine-readable results. The numerical specification is Accurate Models v4 [18]; subnormal input decoding is reconciled with MATLAB Tensor Core v0.5 [27]. The original approximately 500-line CUDA prototype and earlier SMT experiments are historical evidence and are not certified as implementations or proofs of the revised raw-product model. Revised GPU code, compiler/opcode records, and machine-checked proofs remain future artifacts.

REFERENCES

- [1] “Parallel Thread Execution ISA Version 8.4,” Mar 2024. [Online]. Available: https://docs.nvidia.com/cuda/archive/12.4.0/pdf/ptx_isa_8.4.pdf
- [2] “cuda-binary-utilities,” Mar 2024. [Online]. Available: https://docs.nvidia.com/cuda/archive/12.4.0/pdf/CUDA_Binary_Uilities.pdf
- [3] <https://www.nvidia.com/en-us/data-center/tensor-cores/>.
- [4] Gabin, D. A. De Medeiros, J. Faj, A. Marathe, and I. Peng, “On the rise of amd matrix cores: Performance, power efficiency, and programmability,” in *2024 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2024, pp. 132–143.
- [5] B. Feng, Y. Wang, G. Chen, W. Zhang, Y. Xie, and Y. Ding, “EGEMM-TC: Accelerating Scientific Computing on Tensor Cores with Extended Precision,” in *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, ser. PPOPP ’21. New York, NY, USA: Association for Computing Machinery, 2021, p. 278–291. [Online]. Available: <https://doi.org/10.1145/3437801.3441599>
- [6] Y. Lu, L. Zeng, T. Wang, X. Fu, W. Li, H. Cheng, D. Yang, Z. Jin, M. Casas, and W. Liu, “AmgT: Algebraic Multigrid Solver on Tensor Cores,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis*, ser. SC ’24. IEEE Press, Nov. 2024. [Online]. Available: <https://doi.org/10.1109/SC41406.2024.00058>
- [7] C. A. Navarro, R. Carrasco, R. J. Barrientos, J. A. Riquelme, and R. Vega, “GPU Tensor Cores for Fast Arithmetic Reductions,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 1, pp. 72–84, Jan. 2021. [Online]. Available: <https://doi.org/10.1109/TPDS.2020.3011893>
- [8] Y. Niu and M. Casas, “Berrybees: Breadth first search by bit-tensor-cores,” in *Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, ser. PPOPP ’25. New York, NY, USA: Association for Computing Machinery, Feb. 2025, p. 339–354. [Online]. Available: <https://doi.org/10.1145/3710848.3710859>
- [9] H. Ootomo and R. Yokota, “Recovering Single Precision Accuracy from Tensor Cores While Surpassing the FP32 Theoretical Peak Performance,” *The International Journal of High Performance Computing Applications*, vol. 36, no. 4, pp. 475–491, 2022. [Online]. Available: <https://doi.org/10.1177/10943420221090256>
- [10] B. Li, S. Cheng, and J. Lin, “tcfft: A fast half-precision fft library for nvidia tensor cores,” in *2021 IEEE International Conference on Cluster Computing (CLUSTER)*, 2021, pp. 1–11.
- [11] A. Haidar, H. Bayraktar, S. Tomov, J. Dongarra, and N. J. Higham, “Mixed-precision iterative refinement using tensor cores on gpus to accelerate solution of linear systems,” *Proceedings of the Royal Society A*, vol. 476, no. 2243, p. 20200110, 2020.
- [12] H. Bayraktar, A. Charara, D. Clark, S. Cohen, T. Costa, Y.-L. L. Fang, Y. Gao, J. Guan, J. Gunnels, A. Haidar, A. Hehn, M. Hohnerbach, M. Jones, T. Lubowe, D. Lyakh, S. Morino, P. Springer, S. Stanwyck, I. Terentyev, S. Varadhan, J. Wong, and T. Yamaguchi, “cuquantum sdk: A high-performance library for accelerating quantum science,” in *2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*, vol. 01, 2023, pp. 1050–1061.
- [13] X. Li, A. Li, B. Fang, K. Swirydowicz, I. Laguna, and G. Gopalakrishnan, “FTTN: Feature-Targeted Testing for Numerical Properties of NVIDIA & AMD Matrix Accelerators,” in *2024*

- IEEE 24th International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*. New York, NY, USA: IEEE, May 2024, pp. 39–46, <https://github.com/LLNL/FTTN.git>. [Online]. Available: <https://doi.org/10.1109/CCGrid59990.2024.00014>
- [14] M. Mikaitis, “Monotonicity of Multi-term Floating-Point Adders,” *IEEE Transactions on Computers*, vol. 73, no. 6, pp. 1531–1543, 2024.
- [15] IEEE, “IEEE Standard for Floating-Point Arithmetic,” *IEEE Std 754-2019 (Revision of IEEE 754-2008)*, pp. 1–84, Jul. 2019.
- [16] M. Fasi, N. J. Higham, M. Mikaitis, and S. Pranesh, “Numerical Behavior of NVIDIA Tensor Cores,” *PeerJ Computer Science*, vol. 7, p. e330, 2021. [Online]. Available: <https://doi.org/10.7717/peerj-cs.330>
- [17] B. Valpey, X. Li, S. Pai, and G. Gopalakrishnan, “An SMT Formalization of Mixed-Precision Matrix Multiplication: Modeling Three Generations of Tensor Cores,” in *NASA Formal Methods Symposium*. Cham: Springer Nature Switzerland, Jun 2025, pp. 360–379.
- [18] F. A. Khattak and M. Mikaitis, “Accurate models of NVIDIA tensor cores,” arXiv:2512.07004v4, 2026, version 4, June 11, 2026. [Online]. Available: <https://arxiv.org/abs/2512.07004v4>
- [19] G. Bohlender and U. Kulisch, “Comments on Fast and Exact Accumulation of Products,” in *Applied Parallel and Scientific Computing*, K. Jónasson, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 148–156.
- [20] J.-M. Muller, N. Brunie, F. de Dinechin, C.-P. Jeannerod, M. Joldes, V. Lefèvre, G. Melquiond, N. Revol, and S. Torres, *Handbook of Floating-Point Arithmetic*, 2nd ed. Springer International Publishing, May 2018.
- [21] T. J. Dekker, “A Floating-point Technique for Extending the Available Precision,” *Numer. Math.*, vol. 18, no. 3, p. 224–242, Jun. 1971. [Online]. Available: <https://doi.org/10.1007/BF01397083>
- [22] D. E. Knuth, *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*, 1st ed. Addison-Wesley, 1969.
- [23] S. Chowdhary and S. Nagarakatte, “Fast shadow execution for debugging numerical errors using error free transformations,” *Proc. ACM Program. Lang.*, vol. 6, no. OOPSLA2, pp. 1845–1872, 2022. [Online]. Available: <https://doi.org/10.1145/3563353>
- [24] C.-P. Jeannerod and P. Zimmermann, “FastTwoSum Revisited,” in *2025 IEEE 32nd Symposium on Computer Arithmetic (ARITH)*. IEEE, 2025, pp. 141–148.
- [25] D. K. Zhang and A. Aiken, “High-performance branch-free algorithms for extended-precision floating-point arithmetic,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC ’25. New York, NY, USA: Association for Computing Machinery, 2025, p. 695–710. [Online]. Available: <https://doi.org/10.1145/3712285.3759876>
- [26] NVIDIA, “NVIDIA Tesla V100 GPU Architecture,” NVIDIA Corporation, whitepaper WP-08608-001_v1.1, Aug 2017. [Online]. Available: <https://images.nvidia.com/content/volta-architecture/pdf/volta-architecture-whitepaper.pdf>
- [27] F. A. Khattak and M. Mikaitis, “MATLAB tensor core,” Software, version 0.5, 2026, tag v0.5, commit bbcf00a27386. [Online]. Available: <https://github.com/north-numerical-computing/MATLAB-tensor-core/tree/v0.5>
- [28] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*. Philadelphia, PA: Society for Industrial and Applied Mathematics, 1996. [Online]. Available: <http://www.ma.man.ac.uk/~higham/asna.html>
- [29] L. Fousse, G. Hanrot, V. Lefèvre, P. Péliissier, and P. Zimmermann, “Mpfpr: A multiple-precision binary floating-point library with correct rounding,” *ACM Trans. Math. Softw.*, vol. 33, no. 2, p. 13–es, Jun. 2007. [Online]. Available: <https://doi.org/10.1145/1236463.1236468>
- [30] B. Hickmann and D. Bradford, “Experimental Analysis of Matrix Multiplication Functional Units,” in *2019 IEEE 26th Symposium on Computer Arithmetic (ARITH)*. New York, NY, USA: IEEE, Jun. 2019, pp. 116–119. [Online]. Available: <https://doi.org/10.1109/ARITH.2019.00031>
- [31] D. Mukunoki, K. Ozaki, T. Ogita, and T. Imamura, “DGEMM Using Tensor Cores, and Its Accurate and Reproducible Versions,” *High Performance Computing*, vol. 12151, pp. 230–248, 2020. [Online]. Available: <https://api.semanticscholar.org/CorpusID:219709187>
- [32] M. Fasi, N. J. Higham, F. Lopez, T. Mary, and M. Mikaitis, “Matrix Multiplication in Multiword Arithmetic: Error Analysis and Application to GPU Tensor Cores,” *SIAM Journal on Scientific Computing*, vol. 45, no. 1, pp. C1–C19, 2023. [Online]. Available: <https://doi.org/10.1137/21M1465032>
- [33] S. Park, J. Kim, and S. Nagarakatte, “Correctly Rounded Math Libraries without Worrying about the Application’s Rounding Mode,” *Proc. ACM Program. Lang.*, vol. 9, no. PLDI, Jun. 2025. [Online]. Available: <https://doi.org/10.1145/3729332>
- [34] D. K. Zhang and A. Aiken, “Automatic Verification of Floating-Point Accumulation Networks,” in *Computer Aided Verification: 37th International Conference, CAV 2025, Zagreb, Croatia, July 23-25, 2025, Proceedings, Part III*. Berlin, Heidelberg: Springer-Verlag, 2025, p. 215–237. [Online]. Available: https://doi.org/10.1007/978-3-031-98682-6_12
- [35] P. Kornerup, V. Lefevre, N. Louvet, and J.-M. Muller, “On the Computation of Correctly Rounded Sums,” *IEEE Transactions on Computers*, vol. 61, no. 3, pp. 289–298, 2012.
- [36] S. Boldo and G. Melquiond, “Emulation of a FMA and Correctly Rounded Sums: Proved Algorithms Using Rounding to Odd,” *IEEE Transactions on Computers*, vol. 57, no. 4, pp. 462–471, 2008.
- [37] S. Boldo, D. Gallois-Wong, and T. Hilaire, “A

Correctly-Rounded Fixed-Point-Arithmetic Dot-Product Algorithm,” in *ARITH 2020 - IEEE 27th Symposium on Computer Arithmetic*. Portland, United States: IEEE, Jun 2020, pp. 9–16.

- [38] W. Kahan, “Pracniques: Further Remarks on Reducing Truncation Errors,” *Commun. ACM*, vol. 8, no. 1, p. 40, Jan. 1965. [Online]. Available: <https://doi.org/10.1145/363707.363723>